

Semantic-Guided Differentiable Hierarchical Temporal Graph Learning for Log Anomaly Detection

Caizhong Que^{1,2}, Guofang Dong^{1,2*}, Jianming Du³

^{1*}School of Electrical and Information Technology, Yunnan Minzu University, Kunming, 650504, Yunnan, China.

²Yunnan Key Laboratory of Unmanned Autonomous System, Yunnan Minzu University, Kunming, 650504, Yunnan, China.

³College of Computer Science and Technology, Guizhou University, Guiyang, 550025, Guizhou, China.

*Corresponding author(s). E-mail(s): dongguofangyx@163.com;

Contributing authors: 24214038160007@ymu.edu.cn; gs.jmdu24@gzu.edu.cn;

Abstract

Log anomaly detection is challenged by heterogeneous event streams, uncertain workflow boundaries, severe class imbalance, and evolving log templates. We propose a Semantic-Guided Differentiable Hierarchical Temporal Graph framework that jointly learns action-level and workflow-level soft boundaries within predefined outer samples. The framework integrates template, entity, action, state, and continuous-time information through a causal temporal encoder and context-conditioned feature modulation. Nested boundary probabilities are transformed into hierarchical nodes and weighted relations in a heterogeneous temporal graph, enabling anomaly detection from multi-level temporal interactions. Prototype-based normality estimation and class-balanced focal optimization are further introduced to improve robustness under limited anomaly labels. Experiments on five public log datasets, including BlueGene/L, Hadoop Distributed File System, OpenStack, Secure Shell, and Thunderbird, show consistent performance across evaluation settings, with notable improvements on Hadoop Distributed File System and Secure Shell. Additional stress tests and ablation studies demonstrate the effects of boundary learning, structural modeling, and temporal information.

Keywords: log anomaly detection, differentiable segmentation, hierarchical representation learning, heterogeneous temporal graphs

1 Introduction

Large-scale software systems generate high-volume logs that encode runtime states, control-flow events, resource interactions, and recovery behaviors. Because logs are inexpensive to collect and often contain rich textual context, they

remain a central data source for reliability monitoring and security-oriented analytics. Recent studies have expanded log anomaly detection from conventional sequence prediction toward pre-trained language models, self-supervised learning, heterogeneous graph modeling, and retrieval-assisted inference. Nevertheless, the representation of a log sample remains a fundamental

design choice, because anomaly labels are usually attached to sessions, blocks, or windows while the internal workflow structure of those samples is rarely given explicitly [1–5].

A first limitation is the separation between sample construction and anomaly modeling. Deep sequence methods such as DeepLog and LogAnomaly learn sequential regularities from parser-derived events, whereas later Transformer-based models improve contextual representation but still depend on a predefined input sequence [1, 6, 7]. Hierarchical methods have shown that multi-level semantics can improve representation capacity, yet the hierarchy is often induced from fixed linguistic or session structures rather than optimized jointly with the anomaly objective [4, 8, 9]. Consequently, an incorrect internal segmentation can propagate structural errors to subsequent representation learning.

A second limitation is that graph-based approaches typically assume that the graph topology is available before message passing. Recent systems such as LogGT and interpretable spatial-temporal GNNs jointly model event relations, components, and temporal dependencies, demonstrating the value of structural information [5, 10]. However, when the grouping of events into higher-level units is itself uncertain, a fixed graph does not provide a mechanism for the downstream detector to revise the grouping decision. Our objective is therefore not to replace graph learning, but to make a restricted part of the latent structure-sample-internal hierarchical boundaries soft and trainable.

A third limitation concerns data scarcity and distribution shift. Log anomaly datasets are frequently dominated by normal samples, and the normal class can be heterogeneous across components and workloads. Focal and class-balanced objectives are established remedies for skewed supervision, while contrastive and self-supervised methods have recently been used to improve label efficiency [2, 3, 11, 12]. Yet label efficiency does not necessarily follow from a more complex encoder: it depends on whether the additional structure is actually identifiable from the available observations. This distinction is especially important when the benchmark itself is saturated or when entity-level cues are absent.

Motivated by these observations, this paper revisits hierarchical structure learning as a

constrained latent-variable problem inside an already defined outer sample. The proposed Semantic-Guided Differentiable Hierarchical Temporal Graph (SDHTG) framework does not discover complete sessions in an unbounded log stream. Instead, it assumes an outer sample generated by an explicit dataset-compatible protocol, and learns action-level and entity-workflow boundaries within that sample. Boundary probabilities induce soft membership matrices, which simultaneously perform hierarchical pooling and determine the weights of cross-level edges in a heterogeneous temporal graph. This creates a differentiable path from the anomaly objective to the soft structural representation, except for the subsequent discrete graph sparsification step.

The study makes three methodological contributions. First, it formulates sample-internal workflow construction as a constrained latent-structure learning problem and introduces a nested boundary parameterization whose soft probabilities satisfy a position-wise containment relation. Second, it couples these memberships to hierarchical aggregation and heterogeneous temporal graph representation, allowing the anomaly objective to influence the retained soft structural weights. Third, it evaluates the resulting framework under a controlled protocol that separates detection performance from structural identifiability, evidence faithfulness, label scarcity, template evolution, and computational cost.

2 Related Work

2.1 Log parsing and representation learning

Log anomaly detection has historically relied on parsers such as Spell and Drain to convert raw messages into event templates or log keys [13, 14]. Template-based representations simplify downstream sequence modeling, but they also introduce a preprocessing dependency. The empirical study by Khan et al. demonstrated that parsing accuracy itself is not a sufficient predictor of anomaly detection accuracy and highlighted the importance of the distinguishability of parser outputs [15]. This finding is directly relevant to controlled anomaly-detection studies because preprocessing changes can otherwise be

mistaken for model improvements. Recent parser-free or end-to-end methods, including OneLog and LAnoBERT, instead operate closer to the original textual input and seek robustness to unseen or evolving templates [16, 17]. In the present work, Drain is retained as a controlled parser, but it is fitted only on the training partition and frozen thereafter. The methodological focus is therefore not parser innovation but the learning of sample-internal structure under a fixed, auditable preprocessing protocol.

2.2 Sequence and pretrained language models

DeepLog introduced next-event prediction with recurrent sequence modeling, while LogAnomaly incorporated both sequential and quantitative information [6, 7]. LogBERT and LAnoBERT further exploited Transformer-based contextual or masked-language objectives, enabling stronger semantic modeling and improved robustness to certain unseen events [1, 16]. LogFormer extends pretrained representations to cross-domain transfer through a pretrain-and-adapt pipeline [4]. More recent systems such as LogSD and LogEncoder use self-supervised objectives to mitigate the cost of scarce labels and the dominance of frequent messages [2, 3]. These approaches establish strong sequence representations, but the input is still usually a single sequence or a fixed hierarchy. SDHTG addresses the complementary question of whether the internal grouping of events can itself be made a learnable variable.

2.3 Hierarchical and graph-based log modeling

Hierarchical modeling has been explored from several perspectives. HitAnomaly employs hierarchical Transformer representations, while LayerLog models a word-log-log-sequence hierarchy to capture semantics at multiple levels [8, 9]. These works show that log semantics are naturally multi-granular, but their hierarchy is largely determined by predefined levels or grouping procedures. Graph-based methods provide a different mechanism for representing dependencies. Log2vec models enterprise activity as a heterogeneous graph, DeepTraLog combines traces and logs in a graph-oriented anomaly detection

pipeline, and recent methods such as LogGT explicitly represent events, components, and temporal information in heterogeneous graphs [5, 18, 19]. An interpretable spatial-temporal GNN further emphasizes the joint utility of temporal and event-similarity relations for anomaly detection and explanation [10]. SDHTG differs in the structural interface: the weights of event-to-action and action-to-workflow edges are derived from soft boundary memberships rather than fixed grouping rules.

2.4 Boundary-aware and self-supervised anomaly detection

Boundary-aware modeling is the closest methodological line. BASN learns a soft session boundary with a differentiable reset mechanism and combines boundary learning with contrastive representation learning [20]. SDHTG extends this idea in two ways: it uses two nested boundary fields corresponding to action and entity-workflow levels, and it turns soft memberships into explicit hierarchical nodes and cross-level graph weights. The distinction is important because a soft reset changes recurrent state evolution, whereas a membership-based construction changes the representation space on which the anomaly detector operates. At the same time, the present experiments show that boundary learning on real datasets is not automatically identifiable; this limitation is treated as a result rather than hidden by strong claims.

2.5 Class imbalance, label scarcity, and retrieval-assisted methods

Class imbalance has traditionally been addressed through reweighting, resampling, focal loss, or class-balanced loss [11, 12]. In log anomaly detection, recent self-supervised and contrastive approaches have targeted scarce supervision and representation robustness. LogSD uses self-supervised learning with frequency-based masking, while LogEncoder develops contrastive log representations; retrieval-augmented methods further combine semantic retrieval with anomaly

scoring [2, 3, 21]. Large-language-model-based systems such as LLM-LADE move toward semantic reasoning and explanation generation, reflecting a broader shift from template-centric detection toward knowledge-enriched inference [22]. SDHTG does not rely on an external knowledge source; instead, it investigates whether explicit temporal-hierarchical structure provides a useful inductive bias when labels are scarce and log formats evolve.

3 Problem Formulation

Let a log stream be ordered by timestamp. Each event is represented by a template q , an entity identifier u , an action semantic token a , a state semantic token s , and an inter-event gap Δt . For an outer sample $\mathbf{X}$, the events are ordered as $\mathbf{X} = (\mathbf{e}_1, \dots, \mathbf{e}_T)$, with a sample-level anomaly label $y \in \{0, 1\}$. The outer sample is determined before model training by the dataset-compatible protocol (native key, fixed window, or a threshold estimated from the training partition). SDHTG does not connect events across outer samples.

$$\mathbf{e}_t = (q_t, u_t, a_t, s_t, \Delta t_t), \quad \mathbf{X} = (\mathbf{e}_1, \dots, \mathbf{e}_T). \quad (1)$$

Within $\mathbf{X}$, the method introduces two latent boundary fields: an action boundary b_t^A and an entity-workflow boundary b_t^E . A boundary at position t indicates that a new segment begins immediately before event t . The desired soft structure satisfies a containment relation, because an entity-workflow segment is defined as a composition of one or more complete action segments.

$$b_t^A \in \{0, 1\}, \quad b_t^E \in \{0, 1\}, \quad b_t^E \leq b_t^A. \quad (2)$$

The task is to learn model parameters Θ that jointly produce an anomaly score and a soft hierarchical structure from the same outer sample. Importantly, the model is evaluated at the outer-sample level unless an experiment explicitly constructs a ground-truth boundary or event-level target.

4 Proposed SDHTG Framework

4.1 Architecture overview

SDHTG consists of six functional stages: (i) multi-source event encoding, (ii) context-conditioned feature modulation, (iii) nested soft boundary prediction, (iv) differentiable hierarchical aggregation, (v) heterogeneous temporal graph encoding, and (vi) hierarchical anomaly scoring with prototype-based normality estimation. The computation graph links these stages so that the anomaly objective can influence retained soft structural weights.

4.2 Multi-source temporal event encoding

Each categorical field is mapped to a trainable embedding. Unknown values in validation and test data are mapped to an UNK token. For the temporal gap, a logarithmic transform reduces the influence of heavy-tailed intervals, followed by learnable sinusoidal features and an MLP projection.

$$\begin{aligned} \delta_t &= \log(1 + \Delta t_t), \\ \mathbf{r}_t &= [\delta_t; \sqrt{\delta_t + \epsilon}; \sin(\omega_j \delta_t); \cos(\omega_j \delta_t)]_{j=1}^{d_t}, \\ \mathbf{x}_t &= \text{LN}(\text{MLP}_T(\mathbf{r}_t)). \end{aligned} \quad (3)$$

The field embeddings are projected to a common dimension and combined through a learned soft gating function. A unidirectional GRU then produces a left-to-right event representation, ensuring that $\mathbf{h}_t$ depends only on events up to t .

$$\begin{aligned} \alpha_t &= \text{softmax}\left(\text{MLP}_g([\bar{\mathbf{x}}_t^{(1)}; \dots; \bar{\mathbf{x}}_t^{(5)}])\right), \\ \tilde{\mathbf{x}}_t &= \sum_m \alpha_{t,m} \bar{\mathbf{x}}_t^{(m)}, \\ \mathbf{h}_t &= \text{GRU}_{\text{evt}}(\tilde{\mathbf{x}}_t, \mathbf{h}_{t-1}). \end{aligned} \quad (4)$$

4.3 Context-conditioned feature modulation

We encode observable structural-change indicators $d_t^A = \mathbb{I}(a_t \neq a_{t-1})$ and $d_t^E = \mathbb{I}(u_t \neq u_{t-1})$ together with the temporal representation into a low-dimensional strategy state $\mathbf{c}_t$. The strategy

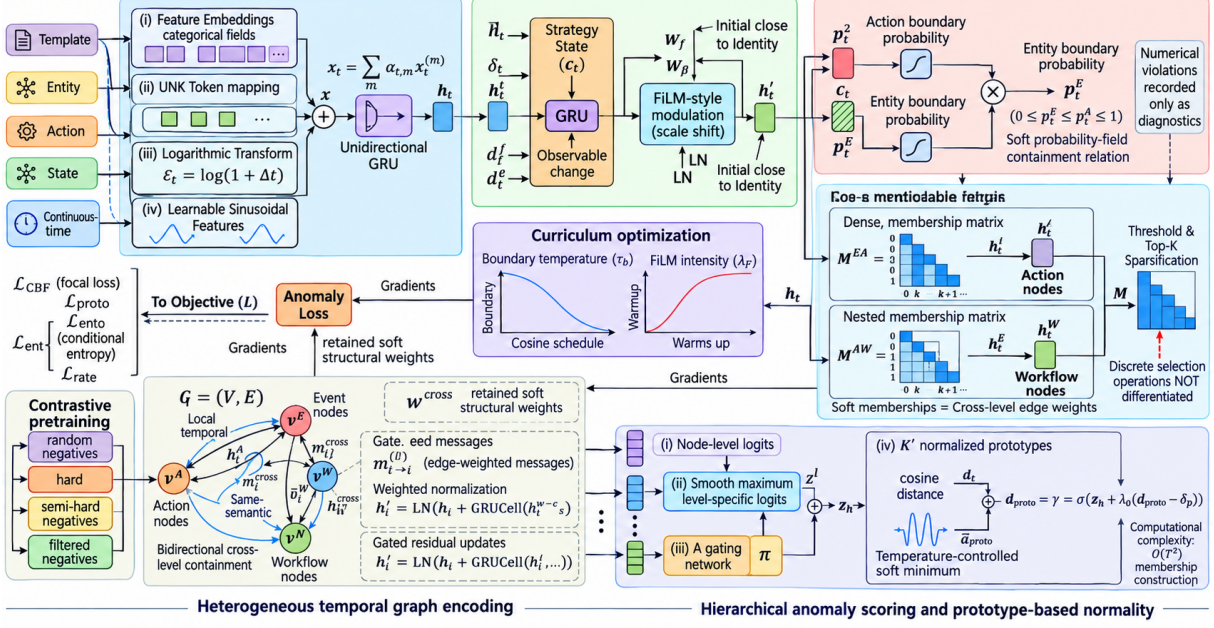


Fig. 1 Overall architecture of SDHTG. The model encodes template, entity, action, state, and continuous-time features; predicts nested soft boundaries; converts the resulting memberships into hierarchical nodes and weighted relations; and aggregates event-, action-, and workflow-level evidence for anomaly scoring. Gradient-based optimization is retained through the soft structural weights, whereas discrete graph sparsification is excluded from gradient propagation.

state generates feature-wise scale and shift parameters. The modulation is initialized close to the identity transformation, which prevents the conditioning mechanism from dominating the encoder during early optimization.

$$\begin{aligned} \mathbf{c}_t &= \text{GRU}_{\text{str}}([\mathbf{h}_t; \delta_t; d_t^A; d_t^E], \mathbf{c}_{t-1}), \\ \mathbf{h}'_t &= \text{LN}\left((1 + \lambda_F \tanh(\mathbf{W}_\gamma \mathbf{c}_t)) \odot \mathbf{h}_t \right. \\ &\quad \left. + \lambda_F \tanh(\mathbf{W}_\beta \mathbf{c}_t)\right). \end{aligned} \quad (5)$$

4.4 Strictly nested soft boundary prediction

Boundary features combine the previous representation, the current representation, their difference, the strategy state, and the observable change indicators. The action boundary probability and a conditional entity boundary probability are predicted independently, after which the entity boundary is defined as their product. This parameterization provides a structural guarantee

without a separate hinge penalty.

$$\begin{aligned} p_t^A &= \sigma\left(f_A(\mathbf{g}_t) + \rho_A \frac{d_t^A - 0.5}{\tau_b}\right), \\ \tilde{p}_t^E &= \sigma\left(f_E(\mathbf{g}_t) + \rho_E \frac{d_t^E - 0.5}{\tau_b}\right). \end{aligned} \quad (6)$$

$$p_t^E = p_t^A \tilde{p}_t^E, \quad 0 \leq p_t^E \leq p_t^A \leq 1. \quad (7)$$

The constraint is deliberately described as a soft probability-field containment relation. It does not imply that a thresholded discrete segmentation is necessarily nested for every possible threshold. The implementation therefore records numerical violations only as a diagnostic quantity rather than optimizing an additional constraint loss.

4.5 Differentiable hierarchical aggregation

For action candidate k , the probability that event t belongs to the segment starting at k is the probability that a boundary occurs at k and no later

boundary occurs before t . The resulting lower-triangular membership matrix is normalized by construction. A second membership matrix is built between action candidates and entity-workflow candidates using the projected entity boundary probabilities.

$$M_{t,k}^{\text{EA}} = p_k^{\text{A}} \prod_{j=k+1}^t (1 - p_j^{\text{A}}), \quad k \leq t, \quad \sum_{k=1}^t M_{t,k}^{\text{EA}} = 1. \quad (8)$$

$$\mathbf{h}_k^{\text{A}} = \frac{\sum_t M_{t,k}^{\text{EA}} \mathbf{h}'_t}{\sum_t M_{t,k}^{\text{EA}} + \epsilon}, \quad \bar{p}_k^{\text{E}} = \frac{\sum_t M_{t,k}^{\text{EA}} p_t^{\text{E}}}{\sum_t M_{t,k}^{\text{EA}} + \epsilon}. \quad (9)$$

$$M_{k,r}^{\text{AW}} = \bar{p}_r^{\text{E}} \prod_{j=r+1}^k (1 - \bar{p}_j^{\text{E}}), \quad \mathbf{h}_r^{\text{W}} = \frac{\sum_k M_{k,r}^{\text{AW}} \mathbf{h}_k^{\text{A}}}{\sum_k M_{k,r}^{\text{AW}} + \epsilon}. \quad (10)$$

The same soft memberships serve as cross-level edge weights. Hence, for an instantiated edge, the anomaly loss can influence the boundary network through the retained membership value. The graph topology itself is sparsified by a threshold and optional top- K rule, and these discrete selection operations are not differentiated.

4.6 Heterogeneous temporal graph construction

Each outer sample is transformed into a heterogeneous graph with three node types: event nodes $\mathcal{V}^{\text{E}}$, action nodes $\mathcal{V}^{\text{A}}$, and workflow nodes $\mathcal{V}^{\text{W}}$. The graph contains three relation families: local temporal edges, same-semantic edges, and bidirectional cross-level containment edges. The number of graph edges is controlled by temporal radius, semantic-neighbor budgets, and retained membership thresholds.

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}), \quad \mathcal{V} = \mathcal{V}^{\text{E}} \cup \mathcal{V}^{\text{A}} \cup \mathcal{V}^{\text{W}}, \quad (11)$$

$$w_{t,k}^{\text{cross}} = M_{t,k}^{\text{EA}}, \quad w_{k,r}^{\text{cross}} = M_{k,r}^{\text{AW}}. \quad (12)$$

For relation type r , a source node j sends a gated, edge-weighted message to target node i . Weighted normalization reduces the scale dependence induced by different node degrees, and a gated residual update preserves the original node

representation.

$$\begin{aligned} \mathbf{m}_{j \rightarrow i}^{(r)} &= w_{ji}^{(r)} [\varphi_r(\mathbf{e}_{ji}) \odot \mathbf{W}_r \mathbf{h}_j], \\ \bar{\mathbf{m}}_i &= \frac{\sum_r \sum_{j \in \mathcal{N}_r(i)} \mathbf{m}_{j \rightarrow i}^{(r)}}{\sum_r \sum_{j \in \mathcal{N}_r(i)} w_{ji}^{(r)} + \epsilon}. \end{aligned} \quad (13)$$

$$\mathbf{h}'_i = \text{LN}(\mathbf{h}_i + \text{GRUCell}(\bar{\mathbf{m}}_i + \mathbf{W}_0 \mathbf{h}_i, \mathbf{h}_i)). \quad (14)$$

4.7 Hierarchical anomaly scoring and prototype-based normality

A detection head produces a logit for each node. Smooth maximum pooling converts node-level evidence into a level-specific logit, while a gating network determines the relative contribution of event, action, and workflow levels. To represent variability among normal samples, K normalized prototypes are learned in the graph-embedding space. The prototype distance is evaluated using cosine distance and a temperature-controlled soft minimum.

$$z^\ell = \tau_d \log \left(\frac{1}{|\mathcal{V}^\ell|} \sum_{i \in \mathcal{V}^\ell} \exp \left(\frac{z_i^\ell}{\tau_d} \right) \right), \quad (15)$$

$$\boldsymbol{\pi} = \text{softmax}(\text{MLP}_{\boldsymbol{\pi}}(\mathbf{c}_{\text{last}})), \quad (16)$$

$$z_{\text{h}} = \sum_{\ell} \pi_{\ell} z^\ell. \quad (17)$$

$$d_k = 1 - \mathbf{g}^\top \mathbf{p}_k, \quad (18)$$

$$d_{\text{proto}} = -\tau_p \log \left(\frac{1}{K} \sum_{k=1}^K \exp \left(-\frac{d_k}{\tau_p} \right) \right). \quad (19)$$

$$z = z_{\text{h}} + \lambda_p (d_{\text{proto}} - \delta_p), \quad (20)$$

$$\lambda_p = \text{softplus}(\eta_p), \quad (21)$$

$$\hat{y} = \sigma(z). \quad (22)$$

The prototype mechanism is interpreted conservatively as prototype-based normality scoring. The experiments show that prototype directions may collapse or become under-utilized on some datasets; therefore the method does not assume that every prototype corresponds to a distinct human-interpretable normal mode.

4.8 Objective and curriculum optimization

The supervised objective uses class-balanced focal loss. Effective-number weighting adjusts the contribution of minority and majority samples before the focal modulation term is applied. Prototype separation and boundary regularization are included in the main optimization, while the contrastive objective is treated as an independent pre-training study rather than an intrinsic component of the principal SDHTG configuration.

$$\begin{aligned}\tilde{\alpha}_c &= \frac{1 - \beta}{1 - \beta^{n_c}}, \\ \alpha_c &= \frac{2\tilde{\alpha}_c}{\tilde{\alpha}_0 + \tilde{\alpha}_1}, \\ \mathcal{L}_{\text{CBF}} &= -\frac{1}{N} \sum_i \alpha_{y_i} (1 - p_{t,i})^\gamma \log(p_{t,i} + \epsilon).\end{aligned}\tag{23}$$

$$\mathcal{L}_{\text{proto}} = \frac{1}{N} \sum_i \left[(1 - y_i) d_i + y_i \max(0, m - d_i) \right].\tag{24}$$

$$\begin{aligned}\mathcal{L}_{\text{ent}} &= -\frac{1}{|\Omega|} \sum_{t \in \Omega} \left[p_t^A \log p_t^A + (1 - p_t^A) \log(1 - p_t^A) \right. \\ &\quad \left. + p_t^E \log p_t^E + (1 - p_t^E) \log(1 - p_t^E) \right].\end{aligned}\tag{25}$$

$$\mathcal{L} = \lambda_{\text{cls}} \mathcal{L}_{\text{CBF}} + \lambda_{\text{proto}} \mathcal{L}_{\text{proto}} + \lambda_{\text{ent}} \mathcal{L}_{\text{ent}} + \lambda_{\text{rate}} \mathcal{L}_{\text{rate}}.\tag{26}$$

The boundary temperature follows a cosine schedule and the FiLM intensity is warmed up during the first training epochs. This curriculum is designed to delay aggressive structural decisions until the event encoder has learned a stable representation. Contrastive pretraining is evaluated separately under random, hard, semi-hard, and semantically filtered negative sampling strategies.

$$\begin{aligned}\tau_b(e) &= \tau_f + (\tau_0 - \tau_f) \frac{1 + \cos(\pi q_e)}{2}, \\ \lambda_F(e) &= \min \left[1, \frac{e + 1}{E_{\text{warm}}} \right].\end{aligned}\tag{27}$$

4.9 Computational complexity

With T events, the dense event-to-action and action-to-workflow membership matrices require $\mathcal{O}(T^2)$ time and memory per sample in the worst case. The current implementation therefore constrains T to 512 and uses length-bucketed batches. Graph sparsification reduces the number of instantiated edges, but does not change the quadratic complexity of the membership construction.

$$C_{\text{membership}} = \mathcal{O}(T^2),\tag{28}$$

$$M_{\text{batch}} = \mathcal{O}(BT^2),\tag{29}$$

$$C_{\text{graph}} \approx \mathcal{O} \left(\sum_{\ell} |\mathcal{V}_{\ell}| (R_{\ell} + K_{\ell}) + (T + |\mathcal{V}^A|) K_m \right).\tag{30}$$

5 Experimental Methodology

5.1 Research questions

- RQ1. Under a controlled input and thresholding protocol, how does SDHTG compare with sequence, convolutional, Transformer, and flat-graph baselines?
- RQ2. Are learned boundaries identifiable when a ground-truth structural boundary is available, and how does performance change when structural cues are deliberately removed?
- RQ3. Which architectural components and imbalance-learning choices contribute consistently, and which should be treated as optional?
- RQ4. How does the structure-aware model behave when labeled anomalies are reduced to 1%, 5%, and 10% of the training labels?
- RQ5. How robust is the model to entity masking, semantic shortcuts, parsing noise, and unseen templates?
- RQ6. Do the extracted boundary and node evidences correspond to measurable changes in anomaly scores?
- RQ7. How do training time, inference latency, memory, and graph size scale with sequence length?

5.2 Datasets and outer-sample protocols

The evaluation uses five public datasets: BGL, HDFS, OpenStack, SSH, and Thunderbird. Dataset statistics were generated directly from the fixed preprocessing pipeline. BGL and Thunderbird are derived from LogHub releases; Thunderbird uses the reported 10M subset. The session-level stratified random split is the primary protocol for the main tables, while temporal splitting is retained as a distribution-shift stress test. Parser fitting, vocabulary construction, threshold selection, and model selection are isolated from the test set.

For BGL and Thunderbird, adaptive idle thresholds are estimated only from the training partition. HDFS events associated with multiple block identifiers are replicated as required by the native block-session protocol, and shared original event identifiers are grouped before partitioning to prevent cross-split duplication. Sequences longer than 512 events are divided into non-overlapping blocks, with labels inherited according to the dataset’s native annotation unit. These preprocessing choices are fixed for the reported experiments.

5.3 Controlled baselines and comparison protocol

The principal baselines are GRU-flat, TCN, Transformer, and GNN-flat. All controlled baselines use the same parsed inputs, semantic fields, sequence lengths, split indices, random seeds, and validation-based threshold selection as SDHTG. GNN-flat preserves the graph representation while removing learnable internal boundaries. Literature-reported numbers are discussed qualitatively because their preprocessing, sessionization, and thresholding protocols differ. This separation is essential: reported performance can vary substantially across preprocessing, sessionization, and thresholding protocols.

5.4 Metrics and statistical analysis

AUPRC is treated as the primary ranking metric because extreme class imbalance can make AUROC comparatively insensitive to the minority class. F1 is reported as the primary threshold-dependent measure, with precision, recall, MCC,

balanced accuracy, and AUROC as secondary measures. Boundary quality is assessed with Boundary F1 under a ± 2 -event matching tolerance. Evidence faithfulness is measured by the change in anomaly logit after removing selected versus random or low-evidence structures.

$$\begin{aligned} F1 &= \frac{2PR}{P+R}, \\ \text{Fid}(k) &= z(\mathbf{X}) - z(\mathbf{X} \setminus R_k), \\ \Delta\text{Fid}(k) &= \text{Fid}(\text{Top-}k) - \text{Fid}(\text{random}). \end{aligned} \tag{31}$$

Five seeds (42, 123, 256, 512, 1024) are used for the principal experiments. Results are summarized by mean $\pm$ standard deviation and 95% confidence intervals. Paired comparisons use the two-sided Wilcoxon signed-rank test with Holm correction, together with Cliff’s δ . Because only five paired seeds are available, inferential claims are based on effect sizes and cross-dataset consistency rather than a binary significance threshold.

5.5 Implementation details

The reported implementation uses PyTorch 2.5.1 and PyTorch Geometric 2.5.3. The event hidden dimension is 256, the strategy dimension is 64, the temporal embedding dimension is 32, and the event GRU has two layers. The graph encoder has two message-passing layers. The default prototype count is $K=8$; semantic-neighbor budgets are 4, 4, and 2 for event, action, and workflow levels. Dropout is 0.10. AdamW is used with a learning rate of 1×10^{-4} and weight decay 1×10^{-5} ; gradient clipping is 5.0. Early stopping is based on validation AUPRC. The reported hardware is an NVIDIA RTX 4060 Laptop GPU with 8 GB memory, AMD Ryzen 7 8845H CPU, Windows 11, Python 3.11.15, and CUDA 11.8.

6 Results and Analysis

6.1 Main detection performance

Three datasets are effectively saturated under the controlled protocol, BGL, OpenStack, and Thunderbird are effectively saturated: AUPRC is close to 1.0 for all evaluated methods. These datasets are therefore more informative for assessing non-degradation and protocol stability than for separating model ranking quality. On HDFS,

Table 1 Dataset statistics and outer-sample construction under the reported preprocessing protocol.

Dataset	Events	Templates	Entities	Outer samples	Anom. rate	Primary split
BGL	4,747,963	350	5,252	287,312	0.1777	60/20/20 session-stratified random
HDFS	11,175,629	45	575,061	575,061	0.0293	60/20/20 grouped temporal
OpenStack	207,809	1,375	12	2,083	0.4364	60/20/20 session-stratified
SSH	655,094	42	1	2,345	0.5650	60/20/20 derived-session
Thunderbird	9,990,882	1,589	5,477	575,817	0.0133	60/20/20 session-stratified random

Table 2 Main anomaly-detection performance under the controlled evaluation protocol. Values are means across the five reported random seeds; detailed secondary metrics are retained for the supplementary material.

Dataset	Method	AUPRC (mean $\pm$ SD)	F1 (mean $\pm$ SD)
BGL	GRU-flat	1.0000 $\pm$ 0.0000	0.9981 $\pm$ 0.0006
	TCN	1.0000 $\pm$ 0.0000	0.9984 $\pm$ 0.0009
	Transformer	1.0000 $\pm$ 0.0000	0.9985 $\pm$ 0.0008
	GNN-flat	1.0000 $\pm$ 0.0000	0.9986 $\pm$ 0.0003
	SDHTG	1.0000 $\pm$ 0.0000	0.9987 $\pm$ 0.0004
HDFS	GRU-flat	0.9984 $\pm$ 0.0016	0.9391 $\pm$ 0.0234
	TCN	0.9988 $\pm$ 0.0009	0.9530 $\pm$ 0.0124
	Transformer	0.9996 $\pm$ 0.0002	0.9439 $\pm$ 0.0228
	GNN-flat	0.9867 $\pm$ 0.0176	0.9056 $\pm$ 0.0375
	SDHTG	0.9974 $\pm$ 0.0019	0.9932 $\pm$ 0.0023
OpenStack	GRU-flat	1.0000 $\pm$ 0.0000	0.9991 $\pm$ 0.0020
	TCN	0.9998 $\pm$ 0.0000	0.9960 $\pm$ 0.0010
	Transformer	0.9999 $\pm$ 0.0001	0.9965 $\pm$ 0.0025
	GNN-flat	0.9999 $\pm$ 0.0001	0.9943 $\pm$ 0.0019
	SDHTG	1.0000 $\pm$ 0.0001	0.9991 $\pm$ 0.0012
SSH	GRU-flat	0.9854 $\pm$ 0.0122	0.9515 $\pm$ 0.0168
	TCN	0.9729 $\pm$ 0.0272	0.9324 $\pm$ 0.0130
	Transformer	0.7484 $\pm$ 0.0905	0.7132 $\pm$ 0.2068
	GNN-flat	0.4171 $\pm$ 0.0609	0.5651 $\pm$ 0.0500
	SDHTG	0.9922 $\pm$ 0.0031	0.9565 $\pm$ 0.0183
Thunderbird	GRU-flat	0.9983 $\pm$ 0.0001	0.9959 $\pm$ 0.0005
	TCN	0.9996 $\pm$ 0.0001	0.9970 $\pm$ 0.0007
	Transformer	0.9994 $\pm$ 0.0006	0.9974 $\pm$ 0.0023
	GNN-flat	0.9998 $\pm$ 0.0001	0.9989 $\pm$ 0.0003
	SDHTG	0.9989 $\pm$ 0.0008	0.9959 $\pm$ 0.0006

SDHTG achieves the highest F1, reaching 0.9932 compared with 0.9530 for TCN and 0.9391 for GRU-flat. However, Transformer obtains the highest AUPRC among the controlled baselines, with 0.9996 compared with 0.9974 for SDHTG. The HDFS result therefore indicates a substantial decision-level advantage without a uniformly dominant ranking function.

On SSH, SDHTG improves over GRU-flat in both AUPRC, from 0.9854 to 0.9922, and F1, from

0.9515 to 0.9565. However, the paired effect size is near zero, so this numerical improvement should not be interpreted as uniformly stable across random seeds. Thunderbird provides an explicit counterexample: GNN-flat achieves an F1 of 0.9989, whereas SDHTG achieves 0.9959. Accordingly, the results support a conditional performance advantage rather than universal superiority. Figure 2

summarizes the absolute F1 values and baseline-relative differences, while Figure 3 separates ranking quality from threshold-dependent decision behavior.

6.2 Structural identifiability and evidence faithfulness

The semi-synthetic experiment provides evidence that the boundary parameterization can recover a known structural seam when informative local cues are available. Under the entity-aligned construction, supervised boundary learning achieves a Boundary F1 of 0.984/0.976, whereas fixed-window alternatives remain below 0.31. However, the same-entity construction deliberately removes the most obvious local entity signal, and performance deteriorates substantially across the evaluated strategies, with most approaches exhibiting weak or near-chance behavior. This contrast indicates that boundary localization is an identifiability problem, not merely a model-capacity problem.

The real-data results provide a stricter qualification. On HDFS and SSH, the evaluated model produces no internal boundaries, while the boundary-score analysis indicates substantial alignment with observable structural priors. Accordingly, the semi-synthetic results validate the boundary mechanism under identifiable conditions, but the real-data results do not establish that the model reliably discovers persistent workflow transitions. Boundary localization should therefore be interpreted as a structure-capable mechanism with explicit applicability conditions rather than as a universally achieved real-data outcome. Figure 4(a) summarizes the identifiability comparison, while Figure 4(b,c) reports the complementary deletion-based evidence analysis.

The deletion analysis provides complementary evidence about the decision relevance of the selected structures. As shown in Figure 4(b,c) and quantified in Table 4, removing top-ranked evidence produces a less negative change in the anomaly logit than removing randomly selected evidence, and the corresponding gap increases with the deletion ratio. At a deletion ratio of 10%, the difference between top-ranked and random removal reaches 0.84. These results indicate that the selected evidence is decision-relevant under the evaluated perturbation protocol.

This analysis should nevertheless be interpreted as a sensitivity-based faithfulness test rather than as a causal explanation. Removing events or structures may create out-of-distribution samples, so the measured change reflects the model’s dependence on the selected evidence rather than the causal importance of those events in the underlying logging process.

6.3 Ablation of architectural and imbalance-learning components

The ablation study does not support a narrative in which every component contributes monotonically. The clearest positive effect is associated with prototype-based normality scoring on HDFS and SSH, whereas the prototype diversity/balance regularizer produces negative changes on both HDFS and SSH. Hierarchical aggregation improves HDFS but is detrimental on SSH, where only one entity is present and the intended entity-level structure is therefore weak. Cross-level message passing is similarly inconsistent across datasets. These results motivate a more modular interpretation of SDHTG: the differentiable hierarchy is the central modeling mechanism, while several graph and prototype refinements are data-dependent and should not be claimed as universally beneficial.

The controlled SSH comparison in Table 6 further shows that loss design and prototype reference configuration affect ranking quality and threshold-dependent performance differently. CB-Focal with the L7 reference achieves the highest reported AUPRC and F1 among the listed CB-Focal configurations, whereas the configuration with the L7b reference yields a lower F1 despite a similar AUPRC. More generally, the improvements in AUPRC do not translate monotonically into improvements in F1. This result emphasizes that imbalance-learning objectives and prototype configurations should be evaluated using both threshold-independent and threshold-dependent metrics.

6.4 Label scarcity and semantic robustness

The reduced-supervision results in Table 7 show that the effect of hierarchical structure under label scarcity is dataset dependent. On BGL, the

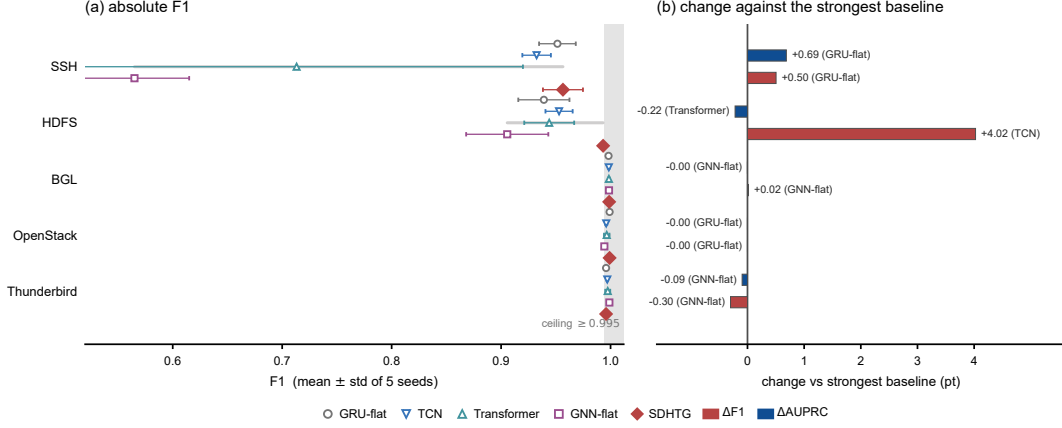


Fig. 2 Main anomaly-detection performance under the controlled protocol. Panel (a) shows F1 means and seed ranges for the four controlled baselines and SDHTG. Panel (b) reports changes relative to the strongest controlled baseline for each dataset and metric. The plots highlight benchmark saturation and the remaining decision-level headroom on HDFS and SSH.

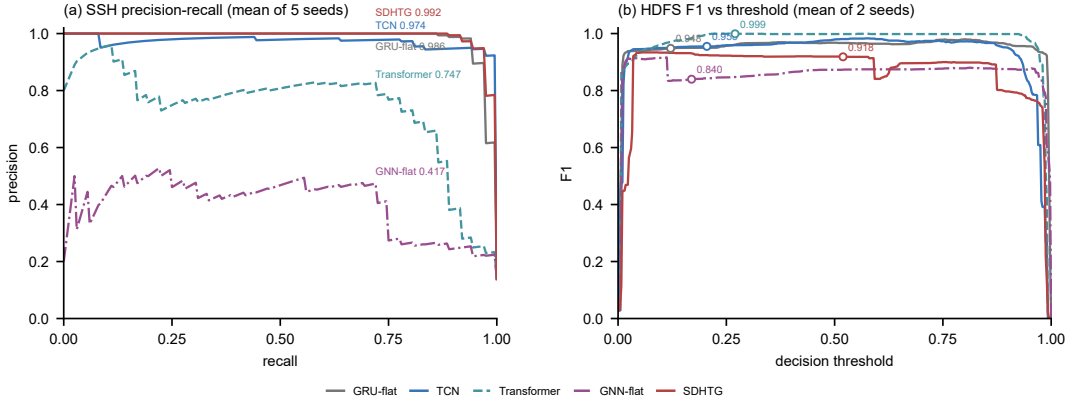


Fig. 3 Ranking quality and threshold-dependent decision behavior. Panel (a) shows precision–recall curves on SSH, while panel (b) shows F1 as a function of the decision threshold on HDFS. The two panels distinguish threshold-independent ranking quality from threshold-dependent calibration and decision behavior.

F1 advantage of SDHTG over GRU-flat increases from 1.09 percentage points under full supervision to 3.87 points at a 1% label ratio. In contrast, SDHTG underperforms GRU-flat on SSH at the 1% label ratio, with decreases of 2.82 AUPRC points and 1.73 F1 points. The results therefore support a conditional interpretation: hierarchical structure can provide a useful inductive bias when the relevant structural cues are observable, but it does not guarantee improved label efficiency when the sample-level structure is weak or poorly aligned with the learned hierarchy.

Entity and semantic shortcut tests provide a complementary robustness check. In BGL, masking approximately 20% of test-session entities reduces F1 by only 0.33 percentage points while leaving AUPRC unchanged. In SSH, removing entity embeddings or boundary-change priors produces only small changes, whereas mapping all test entities to UNK incurs a larger F1 reduction. These findings suggest that entity identity is not the sole decision shortcut, but the effect of semantic fields remains data dependent.

Table 3 Boundary localization performance on the semi-synthetic structural-anomaly benchmark. Boundary F1 is computed from top- k boundary predictions using a ± 2 -event matching tolerance.

Boundary strategy	Entity-aligned	Same-entity
Learned soft boundary (supervised)	0.984/0.976	0.108/0.152
Hard entity change	0.985	0.034
Hard template change	0.654	0.539
Hard action change	0.169	0.105
Fixed window, $w = 4$	0.115	0.335
Fixed window, $w = 8$	0.298	0.335
Fixed window, $w = 16$	0.308	0.363
Random boundary	0.188	0.03–0.54

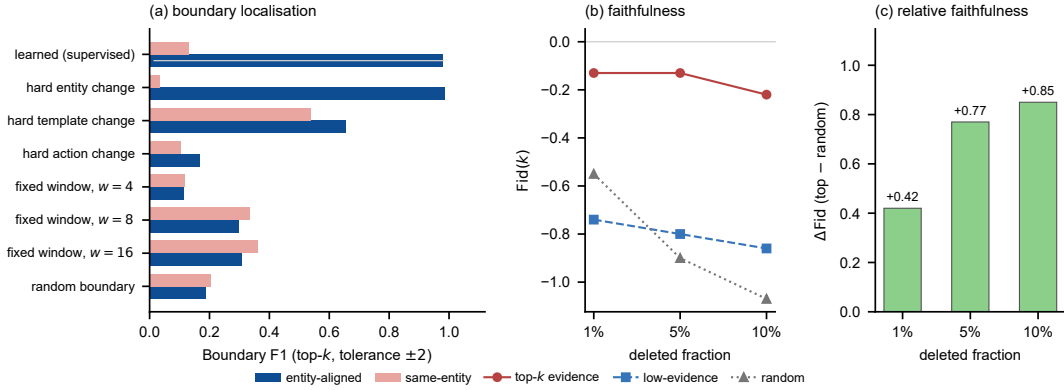


Fig. 4 Structural identifiability and evidence faithfulness. Panel (a) reports Boundary F1 on entity-aligned and same-entity semi-synthetic structural anomalies. Panels (b) and (c) report deletion-based faithfulness and the relative gap between top-ranked and random evidence removal, respectively.

6.5 Template evolution and parsing noise

Temporal splitting of BGL generates a much stronger template-evolution challenge than the main stratified random protocol: the unseen-template rate reaches 79.4% on the test side, compared with 0.004% under the main random split. Under this stress test, both the flat GRU and SDHTG degrade substantially, but the AUPRC gap in favor of SDHTG expands from approximately zero to 15.05 percentage points. This should be interpreted as evidence of improved relative robustness under severe template shift, not as evidence that SDHTG solves open-world template evolution.

The parsing-noise experiment provides a complementary robustness check on SSH. We inject four template-level corruptions—replacement, merge, split, and UNK injection—at a 20%

rewrite rate. For each corruption type, the model is evaluated using the same trained weights and the same decision threshold selected on the clean validation set; the threshold is not recalibrated on the corrupted test condition. Across the noisy conditions, AUPRC changes remain within approximately 0.31 percentage points, indicating that ranking quality is relatively stable under the evaluated parsing perturbations. In contrast, F1 changes are mixed in sign, suggesting that threshold-dependent performance is affected by the interaction between score calibration and the altered test distribution. These results support limited robustness to the tested parsing noise, but they do not establish invariance to arbitrary parser errors or distribution shifts.

Table 4 Structural evidence faithfulness on 60 semi-synthetic test samples. $\text{Fid}(\text{Top-}k)$ and $\text{Fid}(\text{random})$ measure the change in anomaly logit after removing top-ranked and randomly selected evidence, respectively. The relative metric compares top-ranked evidence removal with random removal.

Deletion ratio	$\text{Fid}(\text{Top-}k)$	$\text{Fid}(\text{random})$	ΔFid	ΔFid vs. low-evidence
1%	-0.13	-0.55	+0.42	+0.61
5%	-0.13	-0.90	+0.77	+0.67
10%	-0.22	-1.07	+0.85	+0.64

Table 5 Incremental ablation results. Each row adds one component to the preceding configuration under the same data, seeds, and training budget.

Added module	BGL ΔF1	OpenStack ΔF1	SSH ΔF1	HDFS ΔF1	Thunderbird ΔF1
+FiLM	+0.0002	+0.0004	+0.0085	-0.0014	-0.0009
+action boundary+hierarchy	+0.0002	-0.0013	-0.0146	+0.0272	+0.0011
+nested entity boundary	+0.0001	+0.0009	-0.0030	-0.0404	-0.0003
+temporal edges	-0.0002	-0.0013	+0.0031	-0.0523	+0.0003
+semantic edges	-0.0002	+0.0013	-0.0030	+0.0004	+0.0001
+cross-level message passing	-0.0002	-0.0009	+0.0001	+0.0217	-0.0002
+normal prototypes	+0.0006	+0.0009	+0.0139	+0.0989	-0.0001
+prototype diversity/balance	-0.0001	-0.0017	-0.0147	-0.0086	-0.0006

Table 6 Controlled comparison of imbalance-learning objectives and prototype regularization on SSH. Values are reported under the fixed controlled SSH protocol; complete seed-level results are provided in the supplementary material.

Loss / prototype setting	AUPRC	F1
BCE, 8 prototypes	0.972	0.933
Weighted BCE, 8 prototypes	0.981	0.952
Focal, 8 prototypes	0.989	0.961
CB-Focal, 8 prototypes	0.992	0.951
CB-Focal, 8 prototypes, L7 reference	0.9922	0.9565
CB-Focal, 8 prototypes, L7b	0.9915	0.9419

6.6 Boundary evidence versus structural priors

The alignment analysis provides a more qualified interpretation of the boundary module. As shown in Figure 5(c), total boundary scores are positively associated with the observable action-change rate, especially on SSH. Consistent with this observation, Table 9 reports a total-score AUC of 1.000 on SSH, while the learned-component AUC is only 0.487, which is close to chance-level discrimination. On HDFS, the learned component performs better but remains weaker than the total score, with AUC values of 0.690 and 0.850, respectively.

Because the internal-boundary rate is 0.000 on both datasets, these results do not support the claim that the model reliably discovers persistent workflow transitions in real logs. Rather, they indicate that the boundary mechanism can exploit explicit structural priors, while its independently learned contribution remains limited under the evaluated conditions.

This distinction is important for interpreting the real-data results. The boundary parameterization is capable of representing hierarchical transitions, but its representational capacity does not imply that meaningful latent boundaries will emerge when local structural cues are weak. The results therefore support a structure-capable boundary mechanism with explicit applicability conditions, rather than a fully learned or unsupervised workflow-segmentation procedure. Figure 5 provides complementary diagnostics of level gating, prototype utilization, and boundary-score alignment.

Figure 6 provides qualitative examples of how the model combines boundary evidence, top- k evidence subgraphs, prototype assignments, and sample-level decisions. The true-positive, false-positive, and false-negative cases illustrate distinct diagnostic patterns and failure modes. Because

Table 7 Detection performance under reduced supervision. The apparent label-efficiency benefit is dataset dependent rather than universal.

Dataset	Label ratio	GRU-flat AUPRC / F1	SDHTG AUPRC / F1	Δ AUPRC	Δ F1
BGL	1%	0.9595 / 0.8991	0.9710 / 0.9378	+1.15	+3.87
	5%	0.9794 / 0.9539	0.9899 / 0.9665	+1.05	+1.26
	10%	0.9851 / 0.9632	0.9963 / 0.9765	+1.12	+1.33
	100%	0.9985 / 0.9851	0.9997 / 0.9960	+0.12	+1.09
SSH	1%	0.9920 / 0.9371	0.9638 / 0.9198	-2.82	-1.73
	5%	0.9923 / 0.9521	0.9924 / 0.9428	+0.01	-0.93
	10%	0.9931 / 0.9573	0.9943 / 0.9671	+0.13	+0.98
	100%	0.9854 / 0.9515	0.9922 / 0.9565	+0.68	+0.50

Table 8 Template-evolution stress test on BGL.

Quantity	Value	Interpretation
BGL unseen-template rate, random split	0.004%	Main protocol is close to in-domain
BGL unseen-template rate, temporal split	79.4%	Strong template-evolution stress test
GRU-flat AUPRC, temporal split	0.5040	Severe degradation
SDHTG AUPRC, temporal split	0.6545	Still degraded, but relatively higher
Relative AUPRC gap	+15.05 pt	Robustness advantage under shift

these are selected examples rather than a randomly sampled evaluation set, they are intended to clarify the form of the model’s diagnostic output rather than to estimate aggregate error frequencies. The quantitative conclusions about evidence relevance therefore rely primarily on the deletion-based analysis in Figure 4 and Table 4.

6.7 Efficiency and scalability

The current implementation is not designed for latency-sensitive streaming deployment. At $T = 512$, SDHTG requires 5756.1 ms per training step and 28.38 ms per inference sample, compared with 75.7 ms and 0.45 ms for TCN. These values correspond to approximately $76\times$ the TCN training time and $63\times$ the TCN inference latency, respectively. SDHTG also requires 9.84 GB of peak memory, compared with 1.07 GB for TCN and 1.76 GB for GRU-flat.

As shown in Figure 7(d), graph construction accounts for approximately 82% of the $T = 512$ training-step time, whereas model forward and backward computation accounts for only 3%. This identifies per-sample graph construction as the dominant measured bottleneck, although the dense membership matrices still contribute to

the overall quadratic complexity. The appropriate current positioning is therefore offline or batch-oriented structural analysis rather than latency-sensitive streaming detection. Figure 7 summarizes the scaling behavior and cost attribution.

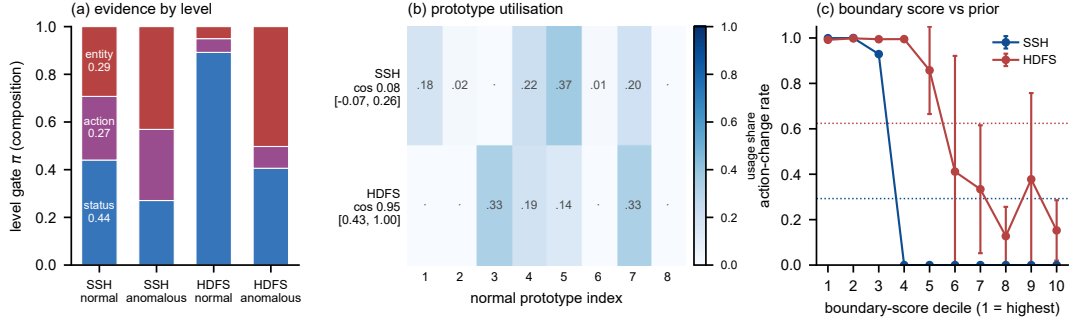
7 Discussion and Limitations

7.1 What the experiments establish

The evidence supports three bounded conclusions. First, soft hierarchical structure can be integrated into a differentiable anomaly-detection pipeline without violating the intended position-wise containment relation. Second, prototype-based normality scoring can provide measurable gains on datasets where the flat baselines leave substantial decision-level headroom. Third, structure-aware representations can improve relative robustness under severe template evolution and, on BGL, under reduced label budgets. These conclusions are narrower than a claim that SDHTG universally outperforms sequence or graph baselines.

Table 9 Boundary-score alignment with observable structural-change priors.

Dataset	Total-score AUC	Learned-component AUC	Internal-boundary rate	Top-10% prior lift
HDFS	0.850	0.690	0.000	1.58
SSH	1.000	0.487	0.000	3.42

**Fig. 5** Hierarchical evidence, prototype utilization, and boundary-score alignment. Panel (a) shows the composition of level-gating weights; panel (b) shows normal-prototype utilization; and panel (c) compares boundary-score deciles with the observable action-change prior.

7.2 Identifiability of the learned hierarchy

A latent boundary is only learnable when the observed representation contains information correlated with the underlying transition. The same-entity semi-synthetic experiment exposes this limitation directly. On real HDFS and SSH, the zero internal-boundary solution is consistent with the absence of informative entity transitions. This means that the model should be interpreted as a structure-capable detector, not as a guaranteed workflow segmentation algorithm.

7.3 Prototype collapse and module selection

Prototype health diagnostics reveal direction collapse on HDFS and under-utilization on SSH. Consequently, the prototype term should be described as a flexible normality distance rather than definitive prototype-based normality estimation. Likewise, negative or near-zero effects for several graph and regularization components argue for a modular design and against presenting the architecture as a monolithic collection of universally useful mechanisms.

7.4 External validity and label semantics

The five public datasets cover diverse log sources but do not represent the full range of production logging environments. Several datasets use coarse session-level labels, and long sessions inherit labels when split into 512-event blocks. A positive session label therefore does not imply that every event in the sample is abnormal. The reported event-level evidence should consequently be interpreted as diagnostic evidence rather than ground-truth event attribution unless a dedicated event-level annotation exists.

7.5 Leakage and protocol risks

The protocol isolates parser fitting, semantic-vocabulary construction, idle-threshold estimation, hyperparameter selection, early stopping, and decision-threshold calibration from the test set. This safeguard is important because empirical evidence shows that log-parsing choices can materially affect downstream anomaly detection results [15].

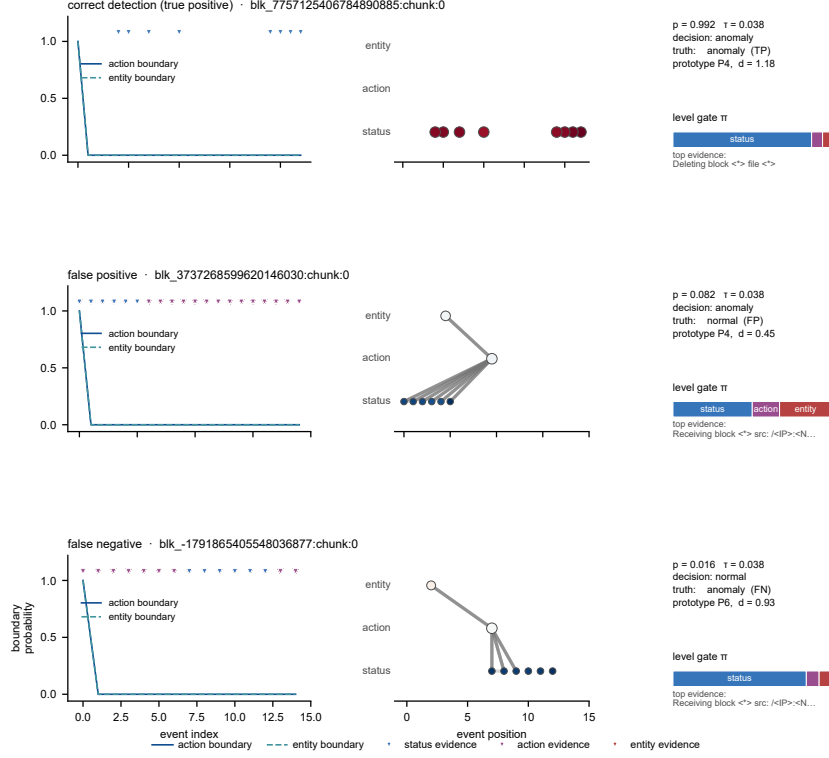


Fig. 6 Representative HDFS cases. Each row contains boundary evidence, a top-k evidence subgraph, and a sample-level decision summary for a true positive, false positive, and false negative, respectively.

Table 10 Operational efficiency of SDHTG across sequence lengths on the reported hardware. Batch size is adjusted to satisfy memory constraints; therefore, the values reflect combined sequence-length and batch-size effects.

T	Batch size	Training (ms/step)	Inference (ms/sample)	Peak memory (GB)	Graph edges
32	512	12828.3	9.26	11.72	942,458
64	128	528.7	2.28	2.23	131,816
128	64	391.6	4.50	2.28	138,136
256	64	986.6	11.38	4.67	288,014
512	64	5756.1	28.38	9.84	607,208

7.6 Interpretability does not imply causality

Boundary probabilities, level weights, prototype assignments, and anomaly subgraphs provide inspectable evidence. They are not, by themselves, causal explanations. The deletion-based faithfulness test strengthens the interpretation by quantifying sensitivity to selected evidence, but deletion can induce out-of-distribution samples. The analysis therefore uses terms such as 'decision evidence', 'structural attribution', or 'faithfulness' rather than 'causal explanation'.

7.7 Reproducibility and implementation caveats

The experimental package records random seeds, configuration values, parser settings, data hashes, and checkpoint states. HDFS also exhibits late-epoch instability for one seed, so validation-based early stopping and best-checkpoint selection are part of the official protocol. Bitwise-identical reruns are not guaranteed across different hardware, CUDA kernels, or software environments.

Table 11 Efficiency comparison at $T = 512$ with the same hidden dimension, batch size, hardware, and warm-up procedure.

Model	Training (ms/step)	Inference (ms/sample)	Peak memory (GB)	Graph edges
GRU-flat	102.7	0.64	1.76	—
TCN	75.7	0.45	1.07	—
Transformer	159.1	0.78	1.90	—
GNN-flat	2143.4	26.28	6.49	606,772
SDHTG	5756.1	28.38	9.84	607,208

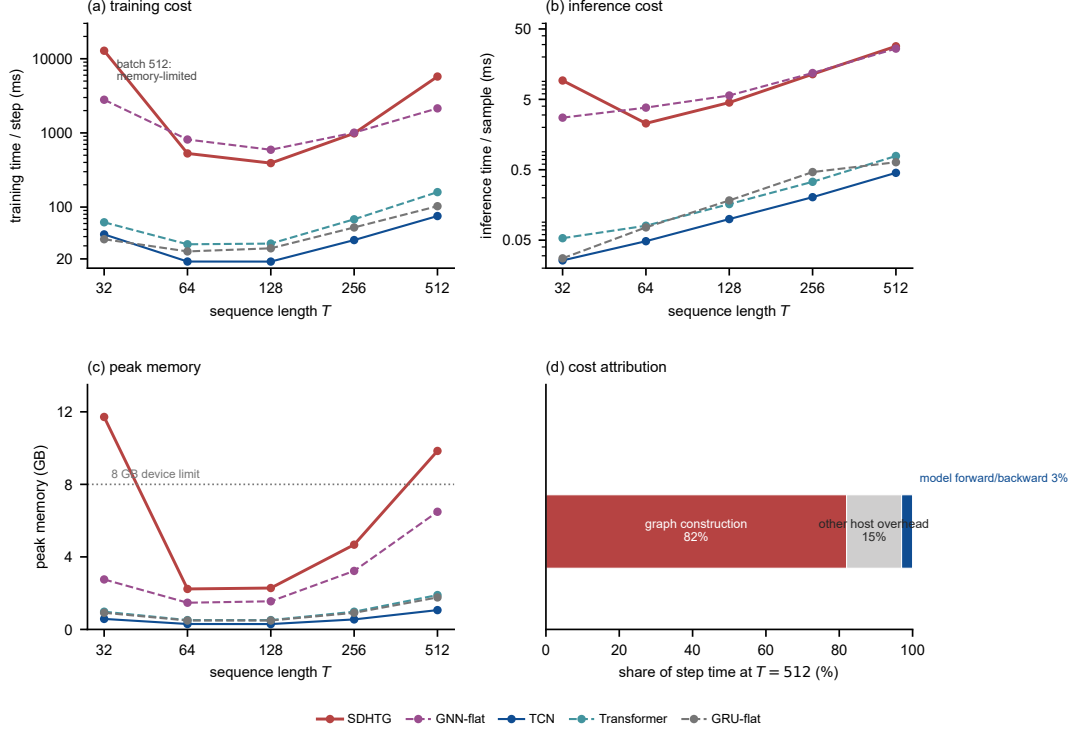


Fig. 7 Computational scaling and cost attribution of SDHTG. Panels (a)–(c) show training time, inference latency, and peak memory as functions of sequence length. Panel (d) decomposes the $T=512$ training step into graph-construction and other overhead.

8 Conclusion

This paper presents SDHTG, a semantic-guided differentiable hierarchical temporal graph framework for log anomaly detection. Rather than treating every internal segment as fixed preprocessing, SDHTG introduces trainable action- and workflow-level soft boundaries within a predefined outer sample and propagates their memberships into a heterogeneous temporal graph. The reported experiments show that the framework can provide useful inductive structure under

specific conditions, especially for HDFS, SSH, reduced-label BGL settings, and severe template evolution. At the same time, the results demonstrate that boundary localization is conditional on observable structural cues, several proposed refinements are not consistently beneficial, prototype representations may collapse, and the current implementation has quadratic membership complexity and high inference cost. These limitations define a concrete path for future work: recursive or sparse membership computation, stronger

open-world semantic representations, continuous-stream state memory, and evaluation with event- or workflow-level ground truth. The resulting formulation is intended as a structure-aware offline anomaly detector with explicit evidence, rather than a claim of universal or fully automatic session discovery.

Supplementary information. If your article has accompanying supplementary file/s please state so here.

Authors reporting data from electrophoretic gels and blots should supply the full unprocessed scans for key as part of their Supplementary information. This may be requested by the editorial team/s if it is missing.

Please refer to Journal-level guidance for any specific requirements.

Acknowledgements. Acknowledgements are not compulsory. Where included they should be brief. Grant or contribution numbers may be acknowledged.

Please refer to Journal-level guidance for any specific requirements.

Declarations

Funding

No external funding was reported for this study.

Competing Interests

The authors declare no conflict of interest.

Ethics approval and consent to participate

Not applicable.

Consent for publication

Not applicable.

Data Availability

The five datasets used in this study are publicly available through the corresponding LogHub releases. Derived preprocessing artifacts, fixed split indices, semantic dictionaries, configuration files, and run manifests are available at [].

Materials availability

Not applicable.

Code Availability

The implementation is available at [????????]. The experiment package records the final configuration, random seeds, checkpoint metadata, and SHA-256 hashes used to reproduce the reported results.

Author Contributions

C.Q.: conceptualization, methodology, software, experiments, formal analysis, visualization, and original manuscript preparation. G.D.: supervision, methodological review, resources, and manuscript revision.

Appendix A Section title of first appendix

An appendix contains supplementary information that is not an essential part of the text itself but which may be helpful in providing a more comprehensive understanding of the research problem or it is information that is too cumbersome to be included in the body of the paper.

References

- [1] Du, M., Li, F., Zheng, G., Srikumar, V.: Logbert: Log anomaly detection via bert. In: 2021 International Joint Conference on Neural Networks (IJCNN), pp. 1–8. IEEE, Shenzhen, China (2021)
- [2] Qi, J., Luan, Z., Huang, S., Fung, C., Yang, H., Li, H., Zhu, D., Qian, D.: Logencoder: Log-based contrastive representation learning for anomaly detection. *IEEE Transactions on Network and Service Management* **20**, 1378–1391 (2023) <https://doi.org/10.1109/TNSM.2023.3239522>
- [3] Xie, Y., Zhang, H., Babar, M.A.: Logsd: Detecting anomalies from system logs through self-supervised learning and frequency-based masking. *Proc. ACM Softw. Eng.* **1**, 2098–2120 (2024) <https://doi.org/10.1145/3660800>

- [4] Guo, H., Yang, J., Liu, J., Bai, J., Wang, B., Li, Z., Zheng, T., Zhang, B., Peng, J., Tian, Q.: Logformer: a pre-train and tuning pipeline for log anomaly detection. In: AAAI'24/IAAI'24/EAAI'24, pp. 135–143. AAAI Press, ??? (2024)
- [5] Wang, P., Zhang, X., Cao, Z., Xu, W., Li, W.: Loggt: Cross-system log anomaly detection via heterogeneous graph feature and transfer learning. *Expert Systems with Applications* **251**(C), 124082 (2024) <https://doi.org/10.1016/j.eswa.2024.124082>
- [6] Du, M., Li, F., Zheng, G., Srikumar, V.: Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1285–1298. Association for Computing Machinery, ??? (2017). <https://doi.org/10.1145/3133956.3134015>
- [7] Meng, W., Liu, Y., Zhu, Y., Zhang, S., Pei, D., Liu, Y., Chen, Y., Zhang, R., Tao, S., Sun, P., Zhou, R.: Loganomaly: unsupervised detection of sequential and quantitative anomalies in unstructured logs. In: *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, pp. 4739–4745. AAAI Press, ??? (2019)
- [8] Huang, S., Liu, Y., Fung, C., He, R., Zhao, Y., Yang, H., Luan, Z.: Hitanomaly: Hierarchical transformers for anomaly detection in system log. *IEEE Transactions on Network and Service Management* **17**(4), 2064–2076 (2020) <https://doi.org/10.1109/TNSM.2020.3034647>
- [9] Zhang, C., Wang, X., Zhang, H., Zhang, J., Zhang, H., Liu, C., Han, P.: Layerlog: Log sequence anomaly detection based on hierarchical semantics. *Applied Soft Computing* **132**(C), 109860 (2023) <https://doi.org/10.1016/j.asoc.2022.109860>
- [10] Xu, R., Li, Y.: Interpretable spatial–temporal graph convolutional network for system log anomaly detection. *Advanced Engineering Informatics* **62**(PC), 102803 (2024) <https://doi.org/10.1016/j.aei.2024.102803>
- [11] Lin, T.-Y., Goyal, P., Girshick, R., He, K., Dollár, P.: Focal loss for dense object detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **42**(2), 318–327 (2020) <https://doi.org/10.1109/TPAMI.2018.2858826>
- [12] Cui, Y., Jia, M., Lin, T.-Y., Song, Y., Belongie, S.: Class-balanced loss based on effective number of samples. In: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9260–9269. IEEE, Long Beach, CA, USA (2019). <https://doi.org/10.1109/CVPR.2019.00949>
- [13] Du, M., Li, F.: Spell: Streaming parsing of system event logs. In: *2016 IEEE 16th International Conference on Data Mining (ICDM)*, pp. 859–864. IEEE, Barcelona, Spain (2016). <https://doi.org/10.1109/ICDM.2016.0103>
- [14] He, P., Zhu, J., Zheng, Z., Lyu, M.R.: Drain: An online log parsing approach with fixed depth tree. In: *2017 IEEE International Conference on Web Services (ICWS)*, pp. 33–40. IEEE, Honolulu, HI, USA (2017). <https://doi.org/10.1109/ICWS.2017.13>
- [15] Khan, Z.A., Shin, D., Bianculli, D., Briand, L.C.: Impact of log parsing on deep learning-based anomaly detection. *Empirical Software Engineering* **29**(6), 139 (2024) <https://doi.org/10.1007/s10664-024-10533-w>
- [16] Lee, Y., Kim, J., Kang, P.: Lanobert: System log anomaly detection based on bert masked language model. *Applied Soft Computing* **146**(C), 110689 (2023) <https://doi.org/10.1016/j.asoc.2023.110689>
- [17] Hashemi, S., Mäntylä, M.: Onelog: towards end-to-end software log anomaly detection. *Automated Software Engineering* **31**(2), 37 (2024) <https://doi.org/10.1007/s10515-024-00428-x>
- [18] Zhang, C., Peng, X., Sha, C., Zhang, K., Fu, Z., Wu, X., Lin, Q., Zhang, D.: Deeptralog: Trace-log combined microservice anomaly detection through graph-based deep learning.

In: 2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE), pp. 623–634. IEEE, Pittsburgh, PA, USA (2022). <https://doi.org/10.1145/3510003.3510180>

- [19] Liu, F., Wen, Y., Zhang, D., Jiang, X., Xing, X., Meng, D.: Log2vec: A heterogeneous graph embedding based approach for detecting cyber threats within enterprise. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, pp. 1777–1794. Association for Computing Machinery, New York, NY, USA (2019). <https://doi.org/10.1145/3319535.3363224>
- [20] Ailabouni, F., Román-Gallego, J.-Á., Pérez-Delgado, M.-L., Grande Pérez, L.: Boundary-aware contrastive learning for log anomaly detection. Applied Sciences **16**(7), 3208 (2026) <https://doi.org/10.3390/app16073208>
- [21] Li, W., Wu, Y., Huang, W., *et al.*: System log anomaly detection based on contrastive learning and retrieval augmented. Scientific Reports **15**, 38370 (2025) <https://doi.org/10.1038/s41598-025-22208-7>
- [22] Zhang, Z., Li, S., Zhang, L., Ye, J., Hu, C., Yan, L.: Llm-lade: Large language model-based log anomaly detection with explanation. Knowledge-Based Systems **326**(C), 114064 (2025) <https://doi.org/10.1016/j.knosys.2025.114064>